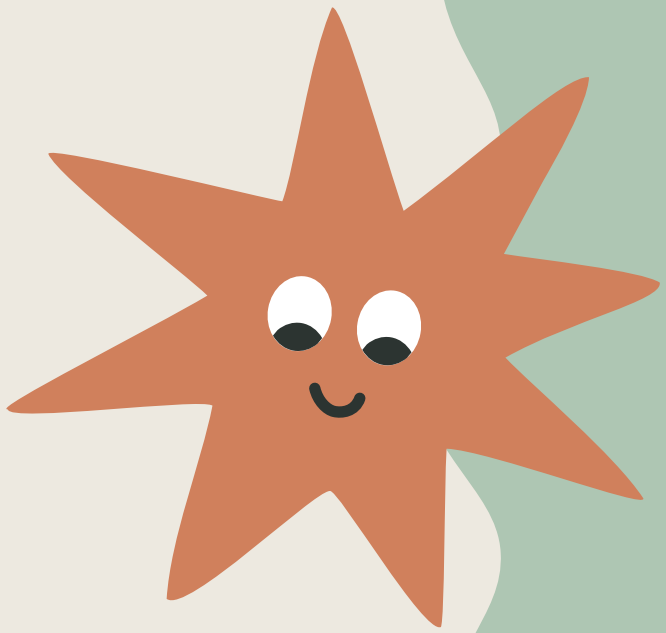
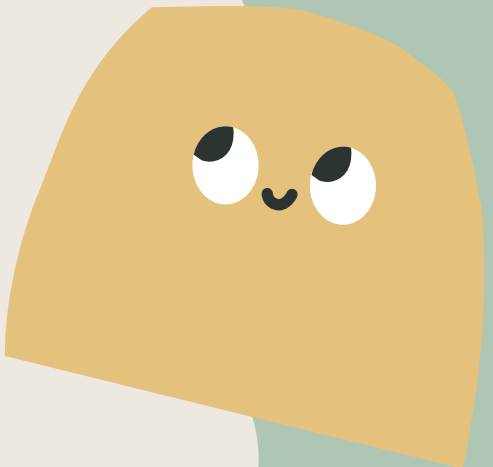
Functions in Python

Lecture 10





Learning Objectives

➢By the end of this lecture, students should be able to:

- Understand why functions are needed
- Define functions using def
- Call functions
- Understand the difference between parameters and arguments
- Return values from functions
- Use multiple parameters
- Use default parameters
- Build reusable programs using functions
- Apply functions to practical and AI-related problems

Prepared By: Ehtisham Rasheed

What is a Function?

- A **function** is a reusable block of code that performs a specific task.
- Instead of writing the same code repeatedly, we write it once inside a function and call it whenever needed.
- For example, suppose we need to calculate the square of a number many times.
- Without a function:

```
number = 5
square = number ** 2
print(square)

number = 8
square = number ** 2
print(square)

number = 10
square = number ** 2
print(square)
```

Prepared By: Fatima Rasheed

What is a Function?

➢With a function:

```
def square(number):
    return number ** 2
```

➢Now:

```
print(square(5))
print(square(8))
print(square(10))
```

➢Much cleaner

Prepared by Mishal Rasheed

Why Do We Need Functions?

1. Reusability
   - Write once and use many times.
2. Organization
   - Large programs can be divided into smaller tasks.
3. Readability
   - Functions make code easier to understand.
4. Maintenance
   - If you need to change the logic, you change it in one place.
5. Modularity
   - Large programs can be divided into independent components.

Prepared By: Ehtisham Rasheed

Defining a Function

- We use the **def** keyword.
- Basic syntax:

```
def function_name():
    # code
```

- Example:

```
def greet():
    print("Hello, students!")
```

- To execute a function, we call it:

```
greet()
```

Prepared by Ehtisham Rasheed

Parameters

- A **parameter** is a variable defined inside the function definition that receives a value.
- Example:

```
def greet(name):
    print("Hello", name)
```

- Here **name** is a parameter
- Now we can call:

```
greet("Ali")
greet("Sara")
greet("Ahmed")
```

Output

```
Hello Ali
Hello Sara
Hello Ahmed
```

Prepared By: Ehtisham Rasheed

Parameter vs Argument

- Look at:

```
def greet(name):
    print("Hello", name)
```

- "name" is the **parameter**
- We can call:

```
greet("Ali")
```

- "Ali" is the **argument**.

- Parameter = variable in function definition
- Argument = actual value passed when calling the function

Prepared By: Ehtisham Rasheed

Function with Multiple Parameters

- A function can have multiple parameters.

```
def add(a, b):
    print(a + b)
```

- Function call:

```
add(10, 20)
```

```
def student_info(name, age, department):
    print("Name:", name)
    print("Age:", age)
    print("Department:", department)

student_info("Ali", 20, "Computer Science")
```

Prepared By Ibtisam Rasheed

Positional Arguments

➢Normally, arguments are assigned according to their position.

```
def student_info(name, age):
    print(name)
    print(age)

student_info("Ali", 20)
```

➢Here:

```
name → "Ali"
age  → 20
```

```
def student_info(name, age):
    print("Name:", name)
    print("Age:", age)

student_info(age=20, name="Ali")
```

Now the order doesn't matter.

This is called **keyword arguments**.

Prepared By: Eh

Return Values

- One of the most important concepts in functions is **return**.
- Consider:

```
def add(a, b):
    return a + b
```

- Now:

```
result = add(10, 20)

print(result)
```

- Output:

```
30
```

Prepared by: Ehtisham Rasheed

Function Returning a String & Boolean

```
def get_greeting(name):
    return "Hello " + name

message = get_greeting("Ali")

print(message)
```

```
def is_even(number):
    return number % 2 == 0

print(is_even(10))
print(is_even(7))
```

```
if is_even(10):
    print("Number is even")
```

Prepared By. Ehtesham Sheed

Function Returning Multiple Values

- Python allows a function to return multiple values.

```
def calculate(a, b):
    total = a + b
    difference = a - b

    return total, difference
```

- Call:

```
total, difference = calculate(20, 5)

print("Total:", total)
print("Difference:", difference)
```

- Output:

```
Total: 25
Difference: 15
```

Python actually returns these values as a tuple.

Prepared by Ehtisham Rasheed

Default Parameters

- We can give a parameter a default value.

```
def greet(name="Student"):
    print("Hello", name)
```

- Now:

```
greet("Ali")
```

```
greet()
```

- Output:

```
Hello Ali
```

```
Hello Student
```

Prepared By: Ehtisham Rasheed

Practical Example — Grade Calculator

```
def calculate_grade(marks):
    if marks >= 80:
        return "A"
    elif marks >= 70:
        return "B"
    elif marks >= 60:
        return "C"
    elif marks >= 50:
        return "D"
    else:
        return "F"
```

```
print(calculate_grade(85))
print(calculate_grade(72))
print(calculate_grade(45))
```

```
A
B
F
```

Function with Dictionary

```python
def get_student_gpa(student):
    return student["gpa"]
```

```python
student = {
    "name": "Ali",
    "age": 20,
    "gpa": 3.45
}

print(get_student_gpa(student))
```

```
3.45
```

```python
def calculate_average(marks):
    return sum(marks.values()) / len(marks)
```

```python
marks = {
    "Python": 85,
    "AI": 91,
    "DSA": 88
}

average = calculate_average(marks)

print("Average:", average)
```

Prepared By:

Homework

- Exercise 1 — Temperature Converter
  - Write a function to convert temperature from Fahrenheit to Celsius
- Exercise 2 — Student Record Analyzer
  - Create a function that receives a student dictionary:

```
student = {
    "name": "Ali",
    "marks": {
        "Python": 85,
        "AI": 92,
        "DSA": 78
    }
}
```

  - The function should return the student's name, average, grade

Prepared by Basham Rasheed